VibeSense Pro

Edge-ML Predictive Maintenance System

Multi-Sensor Vibration Fault Detection on ESP32-S3 with On-Device TinyML

Technical, ML & Backend Report

Harshit Jaiswal
B.Tech ECE, Techno India University — Batch 2023–2027

November 2026

Executive Summary

VibeSense Pro is an end-to-end predictive maintenance system that detects mechanical faults in a rotating DC motor using multi-sensor fusion and edge machine learning. The system captures vibration, acoustic, temperature, RPM, and current signatures from a 6-sensor node built around a Seeed XIAO ESP32-S3, extracts statistical features on-device, and runs a quantized neural network directly on the microcontroller (TensorFlow Lite Micro) to classify the motor's condition in real time — without needing a cloud connection for inference.

The project validated its approach in two stages: first on the CWRU (Case Western Reserve University) bearing-fault benchmark dataset to prove the feature-engineering and modeling pipeline, and second on a self-built physical test rig with induced faults (unbalance, mechanical looseness, and overload) to prove the complete sensor-to-inference-to-cloud pipeline works on real hardware.

Metric	CWRU Benchmark	Own Rig (Motor A, 4-class)
Model	XGBoost	XGBoost / Dense NN (deployed)
Accuracy	88.44%	99.62% / 96.18%
Classes	4 (Normal, Inner, Ball, Outer)	4 (Healthy, Unbalance, Looseness, Overload)
Quantized (INT8) accuracy	—	96.56% (near-zero drop)

The final deployed model runs on-device at 96.56% quantized accuracy across four classes and publishes live predictions over MQTT to a HiveMQ Cloud broker. A Python subscriber stores each message in InfluxDB and a Grafana dashboard visualizes it live. The pipeline was validated end-to-end with live motor tests, including physical fault injection (see the Cloud Backend and Live Validation sections).

System Architecture

Overview

The system follows a sense → extract → infer → transmit pipeline entirely on the edge device, with cloud connectivity for telemetry and dashboarding.

- Sensing: 6 sensors capture vibration, RPM, sound, current, and temperature at the motor.
- Edge inference: raw samples are windowed (50 samples/window) and 12 statistical features are computed on-device.
- On-device ML: a quantized (INT8) neural network, converted to TensorFlow Lite Micro, classifies the window.
- Connectivity and backend: predictions are published over WiFi via MQTT (TLS) to HiveMQ Cloud; a Python subscriber writes them to an InfluxDB time-series database, and a Grafana dashboard visualizes vibration, RPM, and the predicted class live.

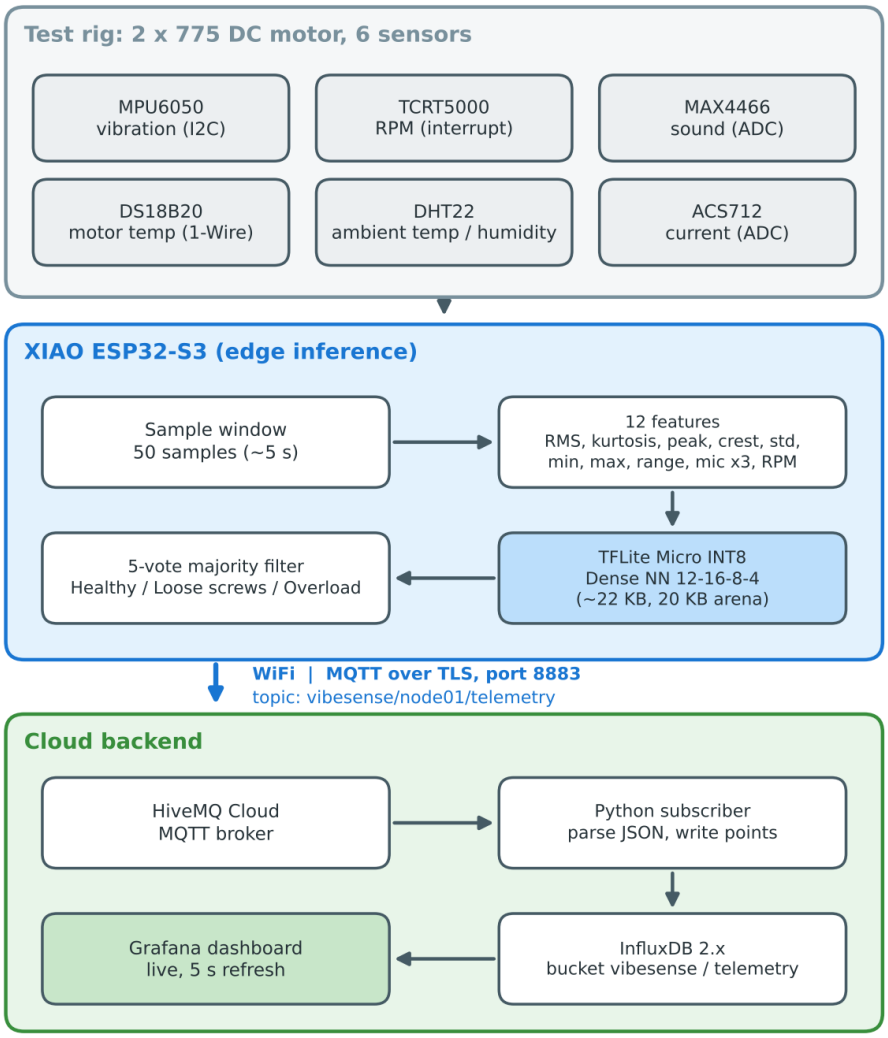


Fig. 1 — System architecture: sensors, on-device TinyML inference, MQTT transport, and the InfluxDB/Grafana backend.

Sensor Suite

Sensor	Signal	Interface	Purpose
MPU6050	3-axis acceleration	I2C (raw register access)	Vibration magnitude — primary fault signal
TCRT5000	IR reflectance pulses	GPIO interrupt	RPM via reflective-strip pulse counting
MAX4466	Analog sound level	ADC	Acoustic signature (motor hum / anomalies)
ACS712	Analog current	ADC	Motor current draw (limited resolution at this load)
DHT22	Ambient temp & humidity	1-Wire	Environmental context
DS18B20	Motor body temperature	1-Wire	Thermal monitoring for overload detection

Physical Build

The two-motor test rig, built on a plywood base with a 775 DC motor, lightweight 3-blade fan, and the full 6-sensor suite per motor, wired to a XIAO ESP32-S3 on a breadboard/perfboard prototype.

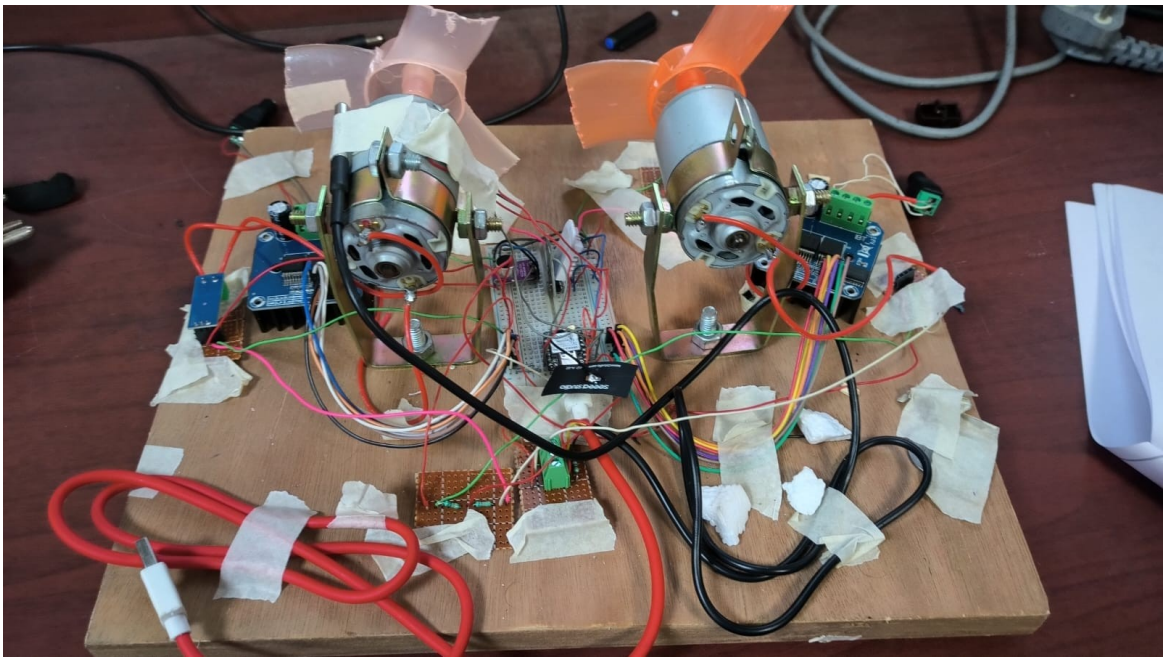


Fig. 2 — Full test rig: two motors, sensor wiring, and the XIAO ESP32-S3 controller board.

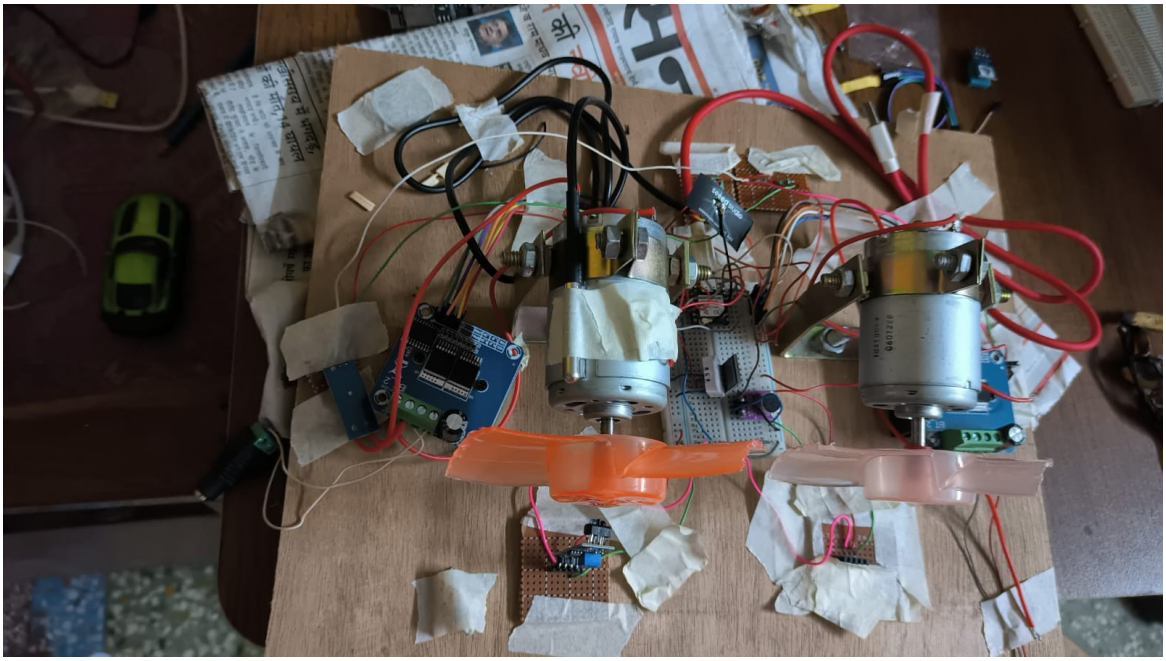


Fig. 3 — Close-up of motor mounting, fan blades, and per-motor sensor breakouts.

Pin Map (XIAO ESP32-S3)

Pin	Sensor
A0	ACS712 (current)
A1	MAX4466 (microphone)
D2	DHT22
D3	DS18B20
D4 / D5	MPU6050 (SDA / SCL)
D6	TCRT5000 (interrupt)

Machine Learning Pipeline — Stage 1: CWRU Benchmark

Before committing to physical hardware, the feature-engineering and classification approach was validated on the industry-standard CWRU bearing fault dataset (16 recordings across Normal / Inner Race / Ball / Outer Race faults, at 0–3 HP loads).

Key Finding: Hardware-Realistic Bandwidth Matters

An initial experiment at the dataset's full 12 kHz sampling rate reached 99.97% accuracy — but this was misleading. The ADXL345/MPU6050 accelerometers available for the physical build sample at roughly 3.2 kHz, well below 12 kHz. Re-running the same simple time-domain features (RMS, kurtosis, peak, crest factor) after resampling to 3.2 kHz collapsed accuracy to 53.97%, because the bearing resonance energy (2–6 kHz) that the easy 12 kHz result was leaning on is lost above the lower Nyquist limit.

Engineering the feature set to include FFT band-energy features tuned to what survives at 3.2 kHz (bands below 1.6 kHz, where the BPFO ≈107 Hz and BPFI ≈162 Hz harmonics still live) recovered performance to 88.44% — the dataset's realistic, hardware-matched result, and the one reported as the project's CWRU benchmark.

CWRU Results Summary

Experiment	Setup	Accuracy
1	12 kHz, single load	100.00%
2	12 kHz, mixed loads	99.97%
3	12 kHz, cross-load	99.97%
4	3.2 kHz resample, v1 features	53.97% (crash)
5	3.2 kHz, v2 features (coarse FFT bands)	87.64%
6	3.2 kHz, v3 features (refined bands) — FINAL	88.44%

At the final hardware-realistic bandwidth, three of the four classes (Normal, Inner Race, Outer Race) reached 100% detection on an unseen load. The Ball fault class was the weak point (23/125 correctly classified) — the physical explanation is that a tumbling ball smears its impact energy across the spectrum, making it resemble the outer-race signature. This is a documented, understood limitation rather than an unexplained gap.

Machine Learning Pipeline — Stage 2: Physical Rig

With the approach validated, a physical test rig was built around two 775 DC motors, each driving a lightweight 3-blade fan, mounted with the 6-sensor suite. The final dataset and model were built on Motor A after establishing it gave a more stable, repeatable baseline than Motor B (whose readings were more sensitive to mounting rigidity and power-supply variation).

Fault Classes Induced

Class	How it was induced
Healthy	Baseline — no modification
Unbalance	Coin taped to one fan blade
Looseness	Motor mounting screws partially loosened
Overload	Light continuous friction applied to the shaft/fan

Data Collected

Class	Raw samples	Feature windows
Healthy	7,420	294
Unbalance	3,577	141
Looseness	9,380	373
Overload (2 sessions combined)	6,012	237
Total	26,389	1,045

Feature Set (12 features per window, 50-sample windows)

RMS, kurtosis, peak, crest factor, standard deviation, min, max, range (all from vibration magnitude); mean, standard deviation, max (from acoustic signal); and mean RPM — computed per window and standardized (z-score) before classification.

Model Results — Motor A, 4-Class

Model	Accuracy	Notes
XGBoost (benchmark)	99.62%	Reference model, not deployed (tree-based models are not TFLite-Micro compatible)
Dense Neural Network (float32)	96.18%	Deployment model architecture
Dense Neural Network, INT8 quantized	96.56%	Final on-device model — negligible accuracy loss from quantization

Per-class performance on the quantized model showed strong separation across all four classes, with the Overload class initially weakest (53% recall on a small first data collection) and improving to 90%+ recall once its training data was doubled by collecting a second independent session — a direct, measured example of how additional data volume corrected a specific class-level weakness.

On-Device Deployment (TinyML)

Quantization Pipeline

- Trained Dense Network (Keras/TensorFlow) converted to TensorFlow Lite with INT8 post-training quantization.
- Representative dataset (100 training samples) used for calibration.
- Converted model exported as a C byte array (21.9 KB) via xxd and embedded directly into the Arduino firmware.
- TensorFlow Lite Micro (Chirale_TensorFlowLite library) runs the interpreter on-device with a 20 KB tensor arena.

On-Device Inference Loop

- Sensors are sampled continuously; a 50-sample sliding window (~5 seconds) is buffered per cycle.
- On window completion, 12 features are computed in C and standardized using the training-set mean/standard-deviation, embedded as constant arrays.
- The scaled feature vector is quantized to INT8 and passed through the on-device interpreter.
- A 5-prediction majority-vote smooths single-window noise before the result is reported and published.
- A minimum-RPM gate (RPM < 200) suppresses inference while the motor is stationary, avoiding meaningless predictions on idle sensor noise.

Connectivity

The XIAO ESP32-S3 connects to WiFi and publishes each voted prediction as a JSON payload over MQTT (TLS, port 8883) to a HiveMQ Cloud broker, on the topic vibesense/node01/telemetry, approximately every 5 seconds while the motor is running.

Field	Description
dev	Device identifier
ts	Device-relative timestamp (receiver applies real-world time on arrival)
cls_name	Predicted class: Healthy / Unbalance / Looseness / Overload
vib	Vibration magnitude (window mean, g)
rpm	Motor speed (window mean)
mic	Acoustic range (window)
curr	Current sensor reading (currently low-confidence — see Limitations)

Cloud Backend & Live Dashboard

The device publishes its predictions over MQTT; the backend receives that stream, stores it as time-series data, and presents it on a live dashboard. It is deliberately decoupled from the firmware: the only interface between the two sides is the MQTT topic and its JSON payload (described under Connectivity), so either side can change independently as long as the contract holds.

Backend Components

Component	Role	Details
HiveMQ Cloud	MQTT broker	TLS on port 8883; topic vibesense/node01/telemetry; the node publishes about every 5 s while RPM > 200
Python subscriber	Ingestion	Subscribes to the telemetry topic, parses each JSON payload and writes a point to InfluxDB
Python simulator	Testing	Publishes synthetic telemetry so the backend could be developed and tested without the rig
InfluxDB 2.x	Time-series storage	Organization vibesense-org, bucket vibesense, measurement telemetry; local instance (port 8086)
Grafana	Live dashboard	Dashboard "VibeSense Dashboard", InfluxDB data source queried with Flux, 5 s auto-refresh (port 3000)

InfluxDB Data Model

Every MQTT message becomes one point in the telemetry measurement. Tags identify the series; fields hold the numeric readings. Payload keys are renamed to descriptive field names on write (for example vib → vibration, curr → current, cls_name → class_name).

Element	Name	Notes
Measurement	telemetry	All node data lives in one measurement
Tag	device	Node identifier (node01); allows more nodes to be added later
Tag	class_name	Class predicted by the on-device model (for example MotorA_Healthy, MotorA_Overload); gives one series per class so conditions can be compared
Fields	vibration, rpm, current, temperature, humidity	Numeric readings; the bucket also holds motor_temperature and mic_range
Timestamp	server receive time	The device ts is relative to boot, so the real-world time is applied when the message arrives

Grafana Dashboard Panels

Panel	Visualization	Source field
Vibration Level	Time series	vibration, one line per class_name
Motor Speed (RPM)	Gauge	rpm, one gauge per class_name
Current Draw (Amps)	Time series	current
Motor Temperature	Time series	temperature
Humidity (%)	Stat (latest value)	humidity

Each panel filters on device = node01 and refreshes every 5 seconds. Because series are split by class_name, a Healthy run and an Overload run appear next to each other on the same panels, which is how the comparison in Fig. 5 was produced.

Running the Backend

- **InfluxDB and Grafana:** run both locally (ports 8086 and 3000); in InfluxDB create the organization, the vibesense bucket and an API token, then add InfluxDB to Grafana as a Flux data source and import the dashboard JSON from the repository.

- **Subscriber:** supply the broker host, username and password (and the InfluxDB URL and token) through environment variables, then start the subscriber. Credentials are kept out of the repository.
- **Without hardware:** start the simulator to publish test data and watch the panels update.

Backend Limitations

- **Local deployment:** InfluxDB, Grafana and the subscriber run on a laptop, so the dashboard is live only while that machine is running; there is no hosted deployment yet.
- **Panels without data:** the current firmware payload carries vibration, RPM, mic and current only, so the Motor Temperature and Humidity panels show data only once those readings are added to the payload (Motor Temperature showed "No data" during the live test).
- **Current is a placeholder:** the ACS712 value is reserved at 0.0 in the firmware (see Known Limitations), so the Current Draw panel is not meaningful yet.
- **No alerting or retention policy:** the dashboard displays the predicted class through per-class series; there is no dedicated state panel, alert rule or long-term retention setup.

Live Validation

The complete pipeline was validated end-to-end on physical hardware: motor spin-up, live sensor streaming, on-device windowing and feature extraction, quantized-model inference, majority-vote smoothing, WiFi/MQTT publish, and live message receipt confirmed on the HiveMQ Cloud web client — all running without a laptop in the loop for inference.

- Baseline (Healthy) detection was reliable and consistent across repeated live tests.
- Unbalance (coin) and Looseness (loosened mount) faults were clearly distinguishable from baseline in the training/test split.
- Overload detection improved substantially after a second, independent data-collection session — recall rose from roughly 53% to over 90%.

Evidence: Live MQTT Telemetry

Live predictions received on the HiveMQ Cloud web client, published directly from the on-device model while the motor ran with a loosened mount (MotorA_Looseness):



Fig. 4 — Live MQTT messages on topic vibesense/node01/telemetry, showing on-device predictions, vibration, RPM, and acoustic readings arriving every few seconds.

Evidence: Backend Dashboard

The Grafana dashboard reads the telemetry stored in InfluxDB and compares Healthy and Overload conditions side by side on live vibration and RPM panels (panel details in the Cloud Backend section):

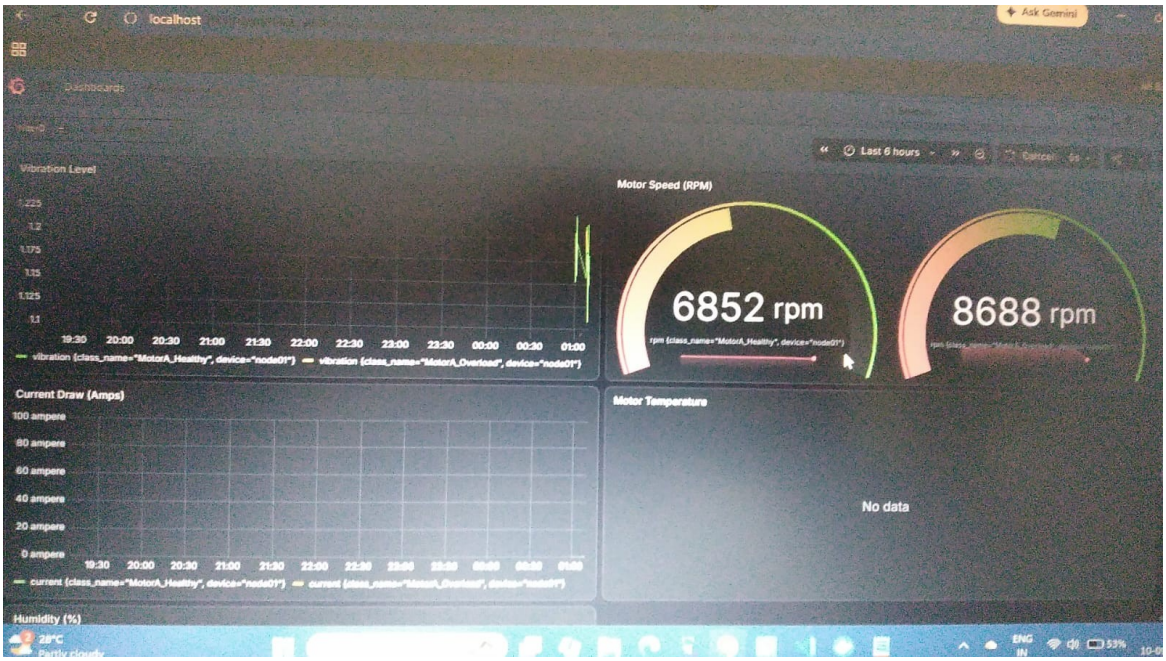
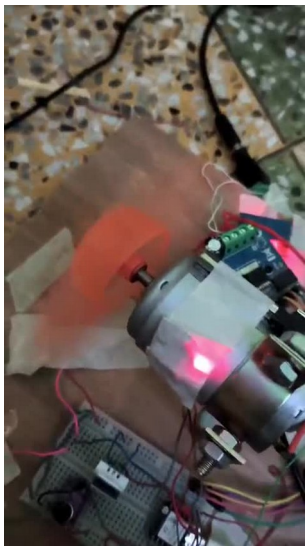
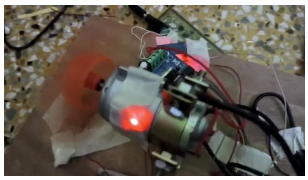


Fig. 5 — Live dashboard: vibration trend and RPM gauges for MotorA_Healthy (6852 rpm) vs. MotorA_Overload (8688 rpm), fed directly from the MQTT stream.

Evidence: Fault Injection in Progress

Frames captured during live operation under each induced condition, confirming the physical fault-injection setup matches the classes reported by the model.

		
Healthy — baseline spin	Looseness — mount partially loosened	Overload — friction applied to shaft

Known Limitations & Honest Observations

- Live (post-training) predictions occasionally misclassify between mechanically similar fault states (e.g., Overload vs. Healthy, Looseness vs. Healthy) more often than the held-out test accuracy alone would suggest — a normal real-world gap between a single training session's held-out split and later live sessions, and consistent with published accuracy figures rather than a code defect.

- RPM measurement is sensitive to TCRT5000 sensor alignment and to session-to-session mounting/power-supply variation; the model still performs strongly using vibration and acoustic features alone (95%+ accuracy without RPM in ablation testing), reducing but not eliminating this dependency.
- The ACS712 current sensor did not reliably separate motor ON/OFF states at this motor's current draw (~0.3–0.8 A), likely below the sensitivity floor of the 5A-rated module used; current is transmitted but not yet a reliable feature.
- WiFi connectivity was initially blocked by a damaged antenna connection on the XIAO board, discovered and repaired mid-project — documented here as a real hardware debugging story, not glossed over.
- Bearing-specific faults (inner/outer race, ball) are validated only on the CWRU benchmark dataset; the physical rig targets mechanical faults that are safe and reversible to induce (unbalance, looseness, overload) rather than destructive bearing damage.

Future Work

- Custom PCB integration of the 6-sensor node, replacing the current breadboard/perfboard prototype.
- Multi-session, multi-condition data collection to further improve live-deployment robustness beyond single-session validation accuracy.
- SHAP-based explainability analysis on the deployed feature set.
- Envelope analysis (Hilbert transform) to address the CWRU Ball-fault weak spot.
- Hosted deployment of the backend (always-on subscriber, InfluxDB and Grafana), a dedicated fault-state panel, alert rules and a retention policy on top of the working live dashboard.
- Higher-sensitivity current sensing (lower-rated ACS712 variant or shunt-based sensing) for reliable current-based features.

Conclusion

VibeSense Pro demonstrates a complete, working edge-ML predictive maintenance pipeline: from raw multi-sensor signals, through on-device feature engineering and a quantized neural network, to live MQTT telemetry stored in InfluxDB and shown on a Grafana dashboard — validated first against an academic benchmark (CWRU, 88.44%) and then on physical, self-built hardware with induced mechanical faults (99.62% XGBoost benchmark / 96.56% deployed quantized model). The project's value lies not only in the final accuracy numbers but in the engineering journey: diagnosing why an initial 99.97% result was misleadingly easy, rebuilding the feature set to survive real hardware bandwidth, debugging real sensor and connectivity failures (RPM sensor alignment, ACS712 sensitivity, motor mounting, WiFi antenna), and iteratively improving a weak class through additional targeted data collection.